Data Warehouse Course Hands-on Project 2026

Rag-Based Sales Data Analysis

Yaohui Ma and Junhao Shao

https://github.com/thezmmm/sales-rag

April 26, 2026

1 System Architecture

The system is a Retrieval-Augmented Generation (RAG) pipeline that answers natural-language analytical questions about the Superstore Sales Dataset (9,994 transactions, 2014–2017). The pipeline is composed of five layers: data processing, vector storage, LLM abstraction, RAG orchestration, and a Streamlit-based chat interface. Figure 1 illustrates the end-to-end data flow.

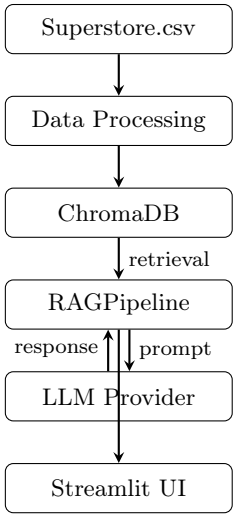

Figure 1: End-to-end system architecture.

Table 1: System component overview.

Layer	Component	Responsibility
Data Proc.	`loader`, `text_conv.`, `chunker`	Load CSV; convert rows/aggregations to natural-language docs; chunk
Vector DB	ChromaDB (×2 cols.)	384-dim embeddings; similarity search; metadata filtering
LLM Layer	`Ollama` `OpenAI`	Unified interface for local/cloud LLMs; streaming; tool-call protocol
RAG Pipeline	`RAGPipeline`	Orchestrate retrieval + generation; `ask()`, `stream()`, `stream_agent()`
Frontend	Streamlit app	Chat UI; sidebar filters; live streaming; agent trace

1.1 Dual-Collection Vector Database Design

Two separate ChromaDB collections are maintained to prevent the large transaction corpus from drowning out the analytically richer summary documents during similarity search:

- `transactions` — one document per sales order row (∼9,994 documents), used for order-level lookups and concrete examples.
- `summaries` — all aggregated documents (∼350 documents), covering monthly, annual, category, regional, seasonal, cross-dimension, statistical, comparative, and trend narratives. Used for trend and ranking questions.

Keeping the collections separate allows the pipeline to apply different top-k values to each and query them independently.

1.2 Two Operating Modes

The `RAGPipeline` supports two retrieval strategies selectable via the `mode` constructor argument:

Direct mode. Both ChromaDB collections are queried unconditionally before each LLM call (top-5 from `summaries`, top-2 from `transactions` by default). The combined context is injected as a single user message along-

side the question. This mode is predictable, fast, and supports true token-by-token streaming via `stream()`.

Agent mode. The LLM receives JSON schemas for two retrieval tools (`search_summaries` and `search_transactions`) and decides autonomously which tools to invoke, with what queries and metadata filters, in a loop of up to six iterations. The loop terminates when the model returns plain text (no tool calls) or the iteration guard is reached. `stream_agent()` yields typed events (`tool_call`, `tool_result`, `answer`) for real-time UI rendering. This mode handles multi-step comparative questions more flexibly than direct mode, at the cost of non-deterministic tool call patterns.

2 Data Preprocessing and Chunking

2.1 Text Representation Strategy

The raw Superstore CSV is transformed into natural-language documents through three modules under `src/data_processing/`. `loader.py` reads the file with `latin-1` encoding (required by the dataset) and parses `Order Date` and `Ship Date` as `datetime64`. `text_converter.py` produces 13 document types across two collections.

2.2 Chunking Strategy

`chunker.py` splits documents into fixed-size character windows with a 50-character overlap to preserve sentence continuity at boundaries. Three chunk sizes were evaluated.

All three sizes produce an identical chunk count because every document in the corpus is shorter than 500 characters — no document is ever split. This is a deliberate consequence of the document design in `text_converter.py`, where each unit covers exactly one transaction, one time period, or one dimension combination. The more consequential granularity decision was therefore made at the document level, not the chunk level: representing each sales order as an individual document determines what the embedding model treats as a single retrievable unit. Chunk size 1,000 was retained as the default for forward compatibility with larger document types.

Table 2: Document types produced by `text_converter.py`.

Type	Content
Transaction	One sentence per order row; annotated with profit sign, discount level, and high-value flag
Monthly	Total sales, profit, margin, order count, avg. discount per calendar month
Annual	Year-level metrics with YoY growth rate and top category/region
Category	Revenue rank, margin, discount per category (3) and sub-category (17)
Regional	Region-level (4) and state-level (49) metrics
Seasonal	Quarter × Year and season aggregates (Winter/Spring/Summer/Fall)
Cross-dimension	Pairwise breakdowns: Region×Category, Year×Category, Year×Region, Segment×Category
Statistical	Four thematic overviews: general, profit, discount, customer
Trend	Multi-year YoY sales and margin narrative
Comparative	Pre-built Technology vs. Furniture and West vs. East comparisons
Seasonal ranking	All quarters and seasons ranked by sales and margin
Region ranking	All four regions ranked by sales and profit
Top performers	Category revenue ranking, top-10 states, sub-category margin ranking

3 Embedding Model, Vector Database, and LLM

3.1 Embedding Model

The project instructions suggest `all-MiniLM-L6-v2` as a baseline. A 13-query retrieval benchmark was conducted to compare it against `BAAI/bge-small-en-v1.5`, covering all four analytical question types required by the rubric. As shown in Table 4, `bge-small` achieves a 33% lower average top-1 cosine distance, indicating stronger semantic alignment for the domain-specific vocabulary of this dataset. Both models share the same embedding dimensionality (384-dim), so the ChromaDB configuration is identical regardless of which is used.

Table 3: Chunk size comparison.

Size	Chunks	Observation
500	10,432	Fine-grained; narrow context per chunk
1,000	10,432	Balanced; selected as default
2,000	10,432	Coarse; may dilute query relevance

Table 4: Embedding model comparison (13-query benchmark).

Model	Params	Dim	Dist.
`all-MiniLM-L6-v2`	22M	384	0.2701
`bge-small-en-v1.5`	33M	384	**0.1796**

Baseline: `all-MiniLM`; Selected: `bge-small`.

3.2 Vector Database

ChromaDB was selected for its native `sentence-transformers` integration and local persistence. The distance metric is cosine similarity (`hnsw:space: cosine`), with scores in $[0, 2]$ where 0 indicates identical vectors. In practice, relevant results consistently appear below 0.5.

Retrieval supports two operations: plain similarity search and metadata-filtered search. Valid filter keys differ by collection — `summaries` supports `type`, `year`, `month`, `quarter`, `season`, `region`, `category`, `sub_category`, `segment`, and `state`; `transactions` supports a subset of these. Filters are exposed as sidebar widgets in the Streamlit UI and as `where` arguments in agent tool calls.

3.3 LLM Selection and Prompt Engineering

The `src/llm/` package decouples the pipeline from any specific model SDK via an `LLMProvider` abstract base class, with two concrete implementations: `OllamaProvider` for local inference and `OpenAIProvider` for any OpenAI-compatible endpoint (OpenAI, Groq, Together AI, etc.). The active provider is selected via the `LLM_PROVIDER` environment variable, requiring no code changes to switch backends.

Two system prompts are used, one per operating mode. In **direct mode**, the LLM is instructed to act as a business intelligence analyst, base answers strictly on the provided context, cite specific numbers, and explicitly state when the context is insufficient rather than fabricating data. In **agent mode**, the prompt additionally specifies tool routing rules: `search_summaries` for aggregate statistics, trends, and rankings; `search_transactions` for concrete order-level examples. This separation reduces unnecessary cross-collection calls and guides the model toward number-backed answers.

Conversation memory retains the last 6 message turns (3 user + 3 assistant), prepended to every prompt. The depth is intentionally shallow to stay within the context limits of smaller local models such as `llama3.2:3b`.

4 Sample Queries and Evaluation

4.1 Retrieval Accuracy

Retrieval quality was evaluated using a 15-case test suite (`retrieval_test.py`) covering all four analytical question types: trend, category, regional, and comparative, plus two transaction-level queries. Four metrics were computed per query: Hit@1 (all expected keywords present in top-1 result), Hit@3 (match within top-3), MRR (mean reciprocal rank), and cosine distance of the top-1 result. Results are reported in Table 5.

Table 5: Retrieval evaluation results (15 test cases, k=5).

Metric	Result	Score
Hit@1	14/15	93.3%
Hit@3	15/15	100.0%
MRR	—	0.9667
Avg dist@1	—	0.1698
High-sim (d < 0.5)	15/15	100.0%

The only Hit@1 failure was *High-discount loss transaction* (case 14): the keyword `loss` appeared in the second result rather than the first (MRR = 0.5, Hit@3 = PASS). All summary-collection queries (trend, category, regional, comparative) achieved 100% Hit@1. Transaction-level queries are inherently harder due to the high document count (∼9,994) and sparse keyword distribution, yielding Hit@1 = 50% but Hit@3 = 100%.

4.2 Analytical Query Demonstrations

Five representative queries were executed against the system using `llama3.2:3b` via Ol-

lama in direct mode. Screenshots of the Streamlit interface are shown in Figures 2–3.

Q1 — Sales trend (2014–2017). *"What is the sales trend over the 4-year period?"* The system retrieved annual summary documents and reported consistent year-over-year growth, citing exact revenue figures for each year.

Q2 — Seasonality. *"Which months show the highest sales?"* Seasonal and quarterly summary documents were retrieved. The model correctly identified Q4 (November–December) as the peak period across all four years.

Q3 — Category revenue. *"Which product category generates the most revenue?"* Category summary documents were retrieved. Technology was correctly ranked first by total revenue; profit margins per category were cited accurately.

Q4 — Regional comparison. *"Which region has the best sales performance?"* Region ranking and regional summary documents were retrieved. The West region was correctly identified as the top performer by both total sales and profit margin.

Q5 — Technology vs. Furniture (agent mode). *"Compare Technology and Furniture sales trends and profitability."* In agent mode, the LLM issued two separate `search_summaries` calls (one per category) before synthesising a comparison. The comparative summary document was also retrieved, providing pre-aggregated per-year figures.

Figure 2: Q1: Sales trend 2014–2017 (direct mode).

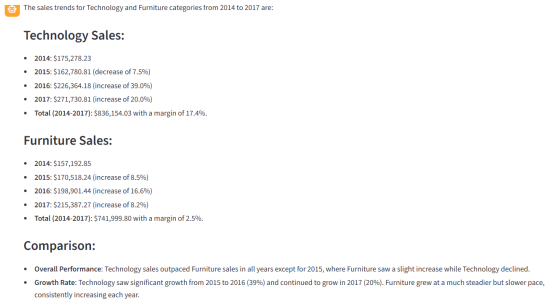

Figure 3: Q5: Technology vs. Furniture comparison (agent mode, showing two `search_summaries` tool calls).

5 Challenges and Solutions

Summary documents overwhelmed by transactions. A single collection caused ∼9,994 transaction documents to consistently outrank the ∼350 summary documents for aggregate queries. Splitting into two independent collections with separate top-k values resolved the issue.

Contradictory metadata filters. The LLM occasionally generated impossible `$and` filters (e.g. `region=West AND region=East`), returning zero results. A defensive parser detects same-key contradictions and falls back to unfiltered retrieval.

Agent mode runaway loops. Smaller models sometimes issued repeated tool calls without converging. A 6-iteration guard was added; on trigger, the model summarises from whatever context it has accumulated.

Embedding model selection. The recommended `all-MiniLM-L6-v2` underperformed on analytical vocabulary. A benchmark identified `BAAI/bge-small-en-v1.5` as superior (avg dist@1: 0.1796 vs. 0.2701) and it was adopted as default.

6 Division of Work

This project was completed as a two-person team. Table 6 summarises the responsibilities of each member. All core architectural decisions — including the dual-collection vector database design, embedding model selection, and RAG pipeline modes.

Table 6: Team division of work.

	Area	Tasks
Yaohui	Data Processing	`loader.py`, `text_converter.py` (13 document types), `chunker.py`
	Vector Database	`store.py`, `build_index.py`, `retrieval_test.py`
	Embedding	Model benchmark and selection
	RAG Pipeline	Direct and agent mode implementation
	Frontend	Streamlit UI; technical report
Junhao	LLM Layer	`base.py`, `ollama.py`, `openai.py`
	Optimisation	Pipeline refinement and debugging
	Demo	Demo preparation and presentation

AI Tool Usage

AI tools were used throughout this project for code generation, debugging, and documentation. The primary tool was Claude Sonnet 4.6 (via Claude Code CLI), covering all implementation phases: data processing, vector database setup, RAG pipeline, and documentation. ChatGPT (gpt-3.5-turbo) was used as the LLM backend for RAG query answering at runtime.

Problem encountered. In the initial implementation, AI-generated code hardcoded API keys directly in `.py` source files, which would have exposed credentials if committed to a public repository. This was identified during code review and resolved by migrating all credentials to a `.env` file loaded at runtime via `python-dotenv`, with `.env` added to `.gitignore`.

Student contributions included all architectural decisions (dual-collection design, embedding model selection, pipeline modes), integration and debugging of AI-generated components, and evaluation of end-to-end output quality. Full details of each usage entry, including specific prompts, problems encountered, running results, and student contributions, are documented at:

https://github.com/thezmmm/sales-rag/blob/main/docs/AI_TOOL_USAGE.md